Graphic Era deemed to be University
DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

EcoRouting Engine: Hazard-Aware Path Optimization and Fill-Rate-Adaptive Waste Scheduling

Student/Team Information

Team Name: *Team #* *PBL Mentor:*	*EcoGraph* *DSCPP-III-2026-T186* *Mr. Navneet Rajput*
Team member 1 (Team Lead)	*Jain, Tejas* *Student ID: 2510011541* *Roll No. 2028228, Sec A* *aispace101@gmail.com* 
Team member 2	*Negi, Apoorv* *Student ID: 2510011088* *Roll No. 2027640, Sec D* *apoorvnegi17@gmail.com* 
Team member 3	*Nigam, Yash* *Student ID: 2510010441* *Roll No. 2028289, Sec H* *nigamyash07@gmail.com* 

PHASE-III PROJECT PROGRESS DESCRIPTION

Project Abstract

In most cities, garbage trucks follow the same streets in the same order every day, even when there is little to collect. The route is set by habit rather than by the shortest path between the depot, the homes and the disposal sites, so fuel, driver time and public money are wasted. There are two more problems. Hazardous waste, like medical sharps, old chemicals and batteries, is often collected along with normal household waste, even though it should be picked up quickly and kept away from homes and markets on the way to disposal. Also, every disposal site is treated as if it fills at the same rate, so a fixed weekly schedule ends up visiting some that are nearly empty and missing others that are overflowing. We wanted to treat the city as what it really is, a graph, and plan collection on top of it.

That is what EcoGraph does. Our C++ backend stores the city as a weighted graph, with every road marked by its zone. It works out the shortest route to collect from a set of homes, decides which homes actually need a visit today based on how fast they fill up, and finds a way to take hazardous waste to disposal while keeping away from residential streets. All of this is available through a REST API, and a PixiJS front end shows the trucks moving along their routes. The app is live at `ecograph-xl8q.onrender.com`.

Updated Project Approach and Architecture

Design. The whole system is one C++17 program, and it serves both the REST API and the web page on the same port. `controllers.cpp` handles HTTP and JSON and keeps the shared data behind a mutex. The actual logic is in separate classes (`Graph`, Dijkstra, `Scheduler`, the waste classifier, the route builder and `Simulation`). They all work on a copy of the city kept in memory, which is loaded from `city.json` and stored as an adjacency list.

Routing. `RouteStrategy` is an abstract base class. Its Priority subclass sets the order of stops, and it always goes to the fullest home first.

Protocol. The front end and the server talk over plain REST with JSON, using 15 endpoints.

Front end. Written in plain JavaScript. Next to the map, it has a day-by-day simulation with rules to simulate a limited number of trucks.

Libraries. We used cpp-httplib for the server, nlohmann/json for JSON and PixiJS to draw the map.

Deployment. We build the Docker image in two stages. The first one compiles the server, and the second keeps only the program, `city.json` and the front-end files, which keeps the image small. It runs as one container on Render, and the page and the API are on the same address.

System overview (diagrams (a)–(h) on the next pages)

1. Every simulated day begins by moving the clock one day forward.

2. A home is due when the days since its last pickup reach capacity / fill rate (kept between 1 and 14 days). The fullest home is served first (d).

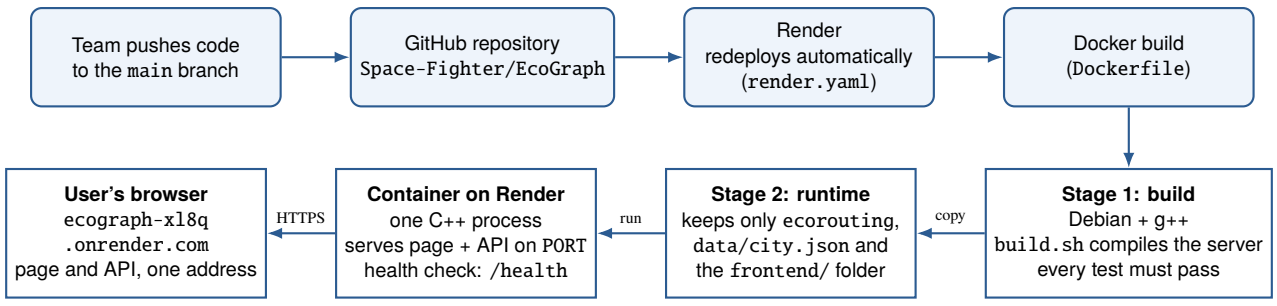
3. Due disposal sites get emptied, hazardous homes go to the hazard trucks (g), and the other homes are sorted and loaded onto trucks before routing (f).

4. Each truck then links shortest paths together in the order of its list:

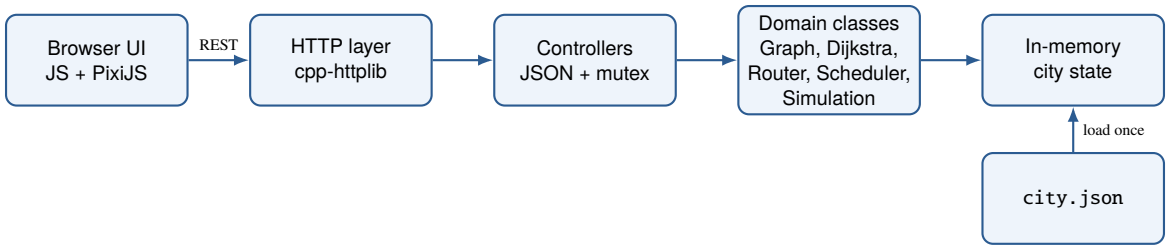
```
current = DEPOT;  visited = { }
for each home in order:
  skip it if in visited
  path = Dijkstra(current → home);  if none, mark unreachable
  empty later stops on the path, add them to visited          (en-route pickup, (e))
  empty home, add to visited;  current = home
then visit the nearest disposal site for each waste type collected
```

5. Every home that was served is marked as collected today. Homes a truck can't reach are reported as failed, and homes that didn't fit on any truck move up the list for the next day.

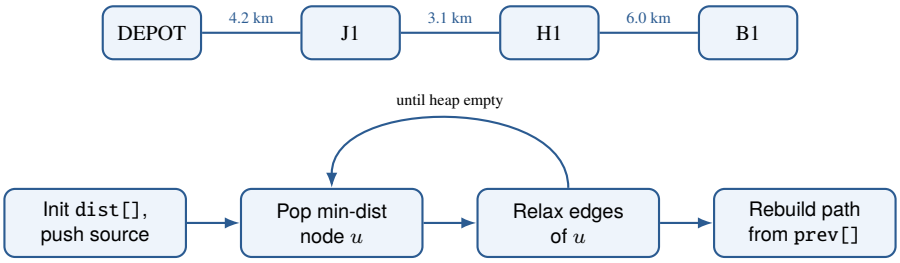
Deployment Setup



(a) System Pipeline



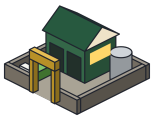
(b) Graph Storage and Dijkstra



(c) Types

These are the same icons the map uses for each type.

Depot



Depot

The depot is where all the trucks are kept. Both normal and hazard trucks start from here every day.

Homes (collection points)

Each point on the map is a small group of buildings, and how it looks depends on its zone.



Residential: houses



Commercial: shops and offices



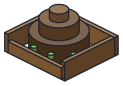
Industrial: factories and warehouses



Hazardous sites: clinics, labs and workshops, marked with a warning sign

Waste types and their disposal sites

The classifier looks for keywords in each home's waste description, and each type of waste goes to its own kind of disposal site.

Disposal site	Waste type	Example keywords	Hazardous?
	General	Anything that matches no other type	No
	Recyclable	plastic, bottle, can, glass, paper, cardboard, newspaper	No
	Compost	food, organic, vegetable, fruit, leaves, peel, garden	No
	Medical	syringe, bandage, needle, blood, gauze, surgical	**Yes**
	Chemical	solvent, acid, battery, paint, pesticide, bleach	**Yes**

Trucks



Normal truck

Collects general, recyclable and compost waste from homes, shops and factories.



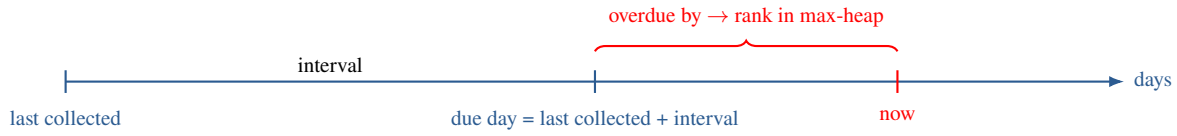
Hazard truck

Collects medical and chemical waste and takes it to a medical or chemical disposal site.

Road types

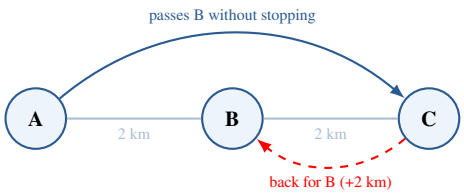
- Arterial: main roads between localities
- Commercial: roads through markets
- Industrial: roads in industrial belts
- Residential: roads through neighbourhoods

(d) Fill-Rate Scheduling

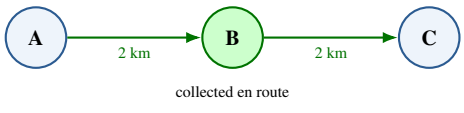


(e) En-Route Pickup

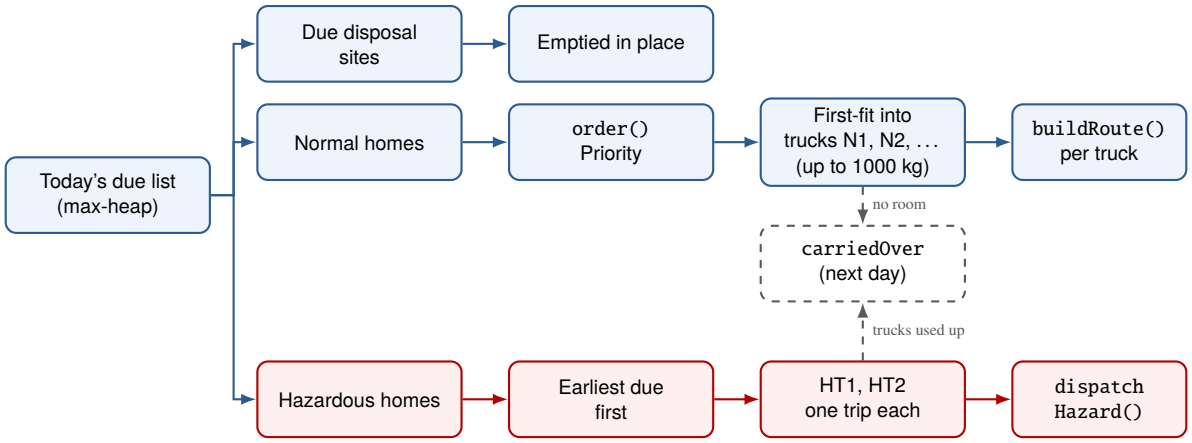
Literal order A → C → B: 6 km



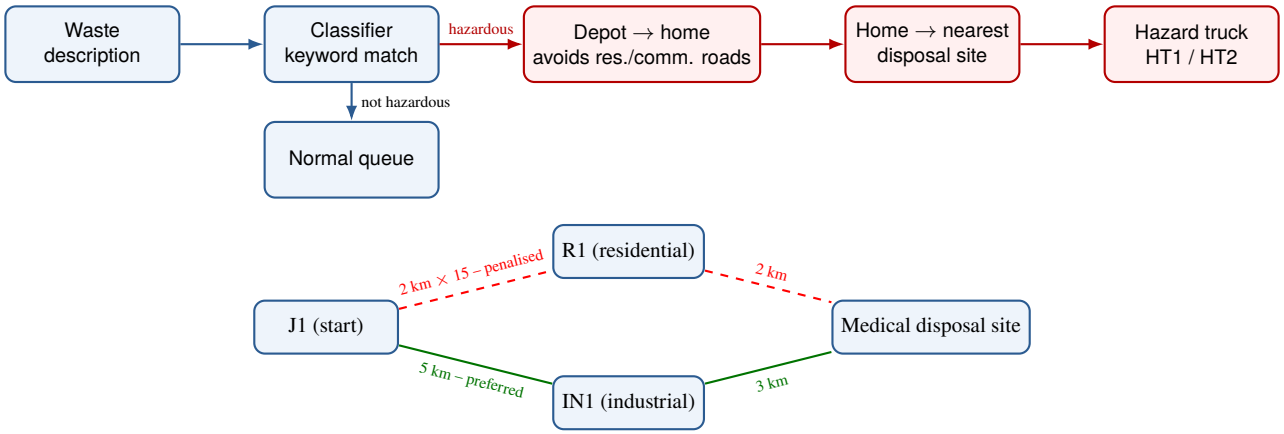
EcoGraph: A → B → C: 4 km



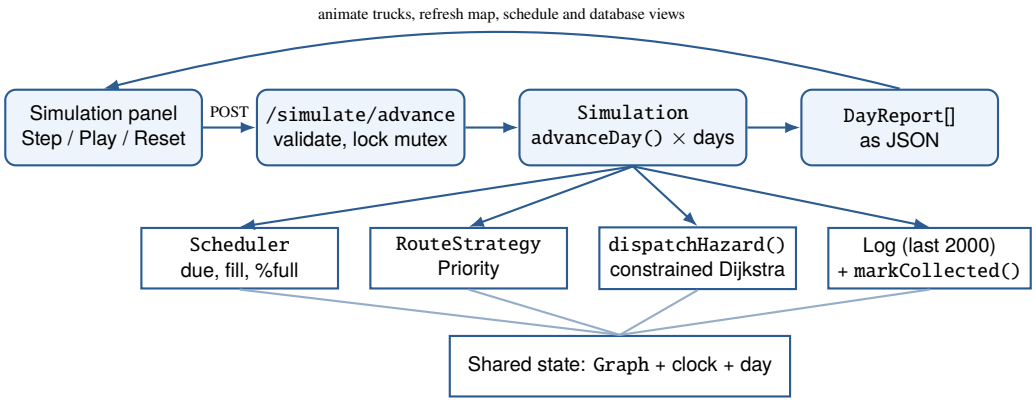
(f) Truck Fleet and Capacity



(g) Automatic Hazard Dispatch and Risk-Weighted Dijkstra



(h) Simulation Architecture



Tasks Completed

Task Completed	Team Member
Built the city as a weighted graph (adjacency list) with a zone on every road, and a loader that reads the Dehradun map (70 nodes, 107 roads) from JSON	Tejas Jain (Lead)
Set up the cpp-httplib server that serves the web page and 15 REST endpoints, with a common error format and a mutex around shared data	Tejas Jain
Wrote Dijkstra's algorithm and a zone-aware version, both able to rebuild the path	Tejas Jain
Wrote the priority route builder (fullest home first), with en-route pickup and unloading at the nearest disposal site	Tejas Jain
Wrote the keyword-based waste classifier and the automatic hazard dispatch to the nearest medical or chemical disposal site, which keeps off residential and commercial roads wherever another road exists	Tejas Jain
Wrote the fill-rate scheduler, which keeps due homes in a max-heap	Tejas Jain
Built the day-by-day simulation: limited trucks, truck capacity, carry-over to the next day and a collection log	Tejas Jain, Apoorv Negi, Yash Nigam
Wrote unit tests for the backend and a check for the city data	Yash Nigam, Apoorv Negi, Tejas Jain
Wrote the README: build steps, folder layout and API reference	Apoorv Negi, Yash Nigam
Built the PixiJS map with truck animation, hazard routes and a legend	Yash Nigam, Tejas Jain, Apoorv Negi
Built the schedule, simulation and database views and the light/dark theme	Tejas Jain, Yash Nigam
Wrote the multi-stage Dockerfile and `.dockerignore`	Yash Nigam, Tejas Jain
Deployed the app on Render as a single container, with a health check on `/health` and automatic redeploys from `main`	Tejas Jain
Set `PORT`, `CITY_FILE` and `FRONTEND_DIR` in the image, and served the page from the same address as the API (`ECO_API_BASE` left empty)	Tejas Jain, Yash Nigam
Added a one-line `docker run` command to the README	Yash Nigam
Added the live demo link to the README	Tejas Jain
Improved the front end	Tejas Jain
Cleaned up the code	Tejas Jain, Apoorv Negi
Wrote the Phase-I, Phase-II and Phase-III reports and presentations	Tejas Jain, Apoorv Negi, Yash Nigam

Challenges/Roadblocks

Road classes beyond residential and commercial. Our first map only had residential and commercial roads, so the hazard truck had no safer road to pick. Real cities also have main roads between localities and industrial areas where disposal sites usually are, so we added *arterial* and *industrial* roads. For a hazard route, a residential road counts as 15 times its length and a commercial road as 10 times. This way the truck stays on main and industrial roads and only goes into a neighbourhood or market when there is no other way.

Banning residential and commercial roads completely. At first, the hazard router would not use residential or commercial roads at all. This only worked because we had connected every hazardous site to a main road in the city file. In reality, a clinic in a market or a lab in a colony can only be reached through its own local street, so with a full ban the truck could not get there and the pickup failed. We replaced the ban with a weighted Dijkstra, where each road's length is multiplied by a risk factor (industrial 1×, arterial 1.5×, commercial 10×, residential 15×). Busy streets are now allowed but cost a lot, so the truck uses them only for the last stretch it can't avoid. Every hazard trip also shows how many km it drove on populated roads (see the figure below).

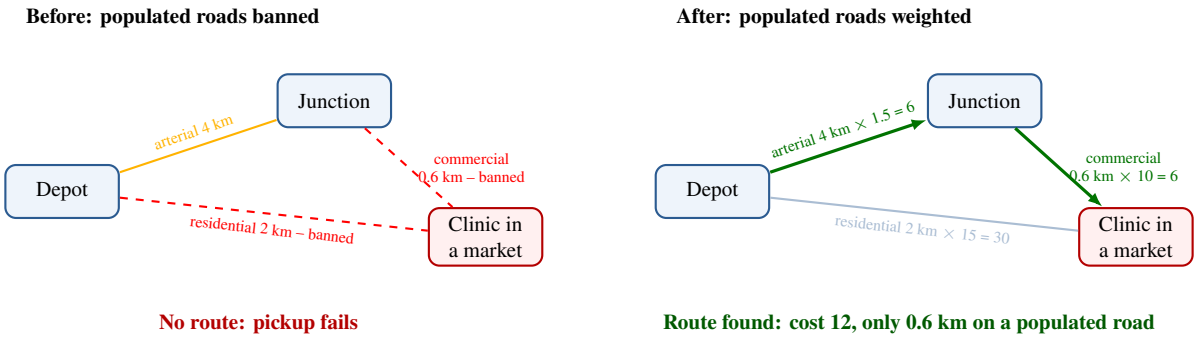
Trucks driving past houses on their own list. In the beginning, the router followed the scheduler's order exactly. If the list was A, C, B and B was on the road from A to C, the truck drove past B to C and then came back for it. We fixed this with en-route pickup. For each part of the route we check if a later stop is on the path, collect it on the way, and skip it when we reach it in the list.

Scheduling that never showed its effect. With unlimited trucks, every due home got served every day and nothing ever overflowed. The priority order only changed the order of stops, not who got served. So we limited each truck to 1000 kg and moved whatever didn't fit to the next day. There was one problem: an overflowing home with more waste than a whole truck could hold would never fit, so an empty truck always takes its first home.

Shared state across requests. cpp-httplib runs requests on several threads at the same time, and routing, collection and the simulation all change the same graph. We put all the shared data behind one mutex so two requests can't clash.

Compiler on Windows. The old MinGW g++ could not build cpp-httplib because it doesn't fully support threads, so we moved to GCC 13.

Hazard routing: full ban vs. weighted roads



Project Outcome/Deliverables

- A C++17 server that loads a city and handles routing, hazard dispatch, waste classification, scheduling and simulation through a REST/JSON API.
- A web page that shows the Dehradun map, animates each truck's route, highlights hazardous trips and lets you move the city ahead one day at a time.
- Unit tests for every main module, and a sample city file that anyone can change.
- A live demo at `ecograph-xl8q.onrender.com`, the README and API docs, and the Phase-II and Phase-III reports and presentations.

Our goal was simple. For any list of homes, the system should find a shorter route than a fixed one, keep hazardous waste away from residential and commercial streets whenever another road exists, and stop sending trucks to disposal sites that are not even close to full.

Progress Overview

We have finished the project. At the end of Phase-II we were roughly 80% done and still had to deploy the app, improve the front end and clean up the code. We completed all three in this phase. Every part we planned now works together, from the city graph and Dijkstra to the classifier, the scheduler, the simulation and the web page, and the same version is live online.

We kept to our plan. We had set deployment for Phase-III back in Phase-I, and that is when we did it. We also added two things we hadn't planned at first: the day-by-day simulation and a limited number of trucks. A single route on a fixed map couldn't show whether our schedule actually kept up over time, so we needed them.

The live link is in the README. What remains is the Phase-III presentation and the final demo.

Codebase Information

Repository: `github.com/Space-Fighter/EcoGraph` **Branch:** `main`

Hosted demo: `https://ecograph-xl8q.onrender.com`

The `server/` folder has the C++ code, headers, tests and `data/city.json`, and the `frontend/` folder has the HTML, CSS and JavaScript files.

Key files

File	What it does
`server/src/main.cpp`	Starts the server; reads the port, city file and front-end folder
`server/src/controllers.cpp`	The 15 REST endpoints: parses JSON, calls the domain classes, guards shared state with a mutex
`server/src/graph.cpp`	City graph: nodes and zone-tagged roads stored as an adjacency list
`server/src/dijkstra.cpp`	Plain and zone-constrained shortest path
`server/src/scheduler.cpp`	Collection interval, due, overdue and % full; max-heap of due locations
`server/src/waste_classifier.cpp`	Keyword rules: medical, chemical, recyclable, compost (else general)
`server/src/router.cpp`	Priority route builder, en-route pickup, nearest disposal site unloading, hazard dispatch
`server/src/simulation.cpp`	Advances days, loads trucks, runs routes and hazard trips, keeps the collection log
`server/src/city_loader.cpp`	Reads `city.json` into the graph
`server/data/city.json`	The Dehradun city: localities, nodes, roads, zones, fill rates
`server/tests/`	Unit test programs (Dijkstra, domain, simulation, generator, city data)
`frontend/js/graph-view.js`	PixiJS map and truck route animation
`frontend/js/schedule-view.js`, `simulation-view.js`, `database-view.js`	Schedule, simulation and database panels
`frontend/js/api.js`	Thin `fetch` wrapper around the REST API
`Dockerfile`, `.dockerignore`, `render.yaml`	Container image and Render deployment settings

Important commits and the modules they added

Commit	Modules / checkpoint
Graph model and Dijkstra	`graph`, `dijkstra` (plain and constrained), Dijkstra tests
REST API server, city data and front end	`scheduler`, `waste_classifier`, `router` (Priority), `controllers`, map view
Day-by-day simulation and database view	`simulation`, database view, simulation tests
Hazard truck dispatch and limited fleet	automatic hazard trucks, truck count and capacity in `simulation` and `router`
City mapped to Dehradun (km/kg units)	real coordinates and localities
Full README with API reference	README, API docs
Schematic locality layout	cleaner map layout of the localities
En-route pickup	en-route collection in `router`
Add Dockerfile (`5ab2551`)	Docker image and `docker run` instructions
Add Render deployment config (`0b8130f`)	`render.yaml`, updates to `Dockerfile` and `.dockerignore`

Testing and Validation Status

Test Type	**Status (Pass/Fail)**	**Notes**
Unit: Dijkstra (`test_dijkstra`)	Pass	Gives the same answers as graphs we solved by hand, and handles a target it can't reach without crashing
Unit: domain (`test_domain`)	Pass	Classifier rules, scheduler intervals kept between 1 and 14 days, priority order and tie-breaks, the zone-aware route avoids a blocked road, hazard split
Unit: simulation (`test_simulation`)	Pass	Trucks never carry more than their capacity, homes left over move to the next day, and hazard trucks work as expected
Unit: generator (`test_generator`)	Pass	New nodes are unique, connected to the roads, reachable and placed inside their locality
Data: city file (`test_city_data`)	Pass	The Dehradun city file has its depot, localities and hazard homes set up correctly
API checks on the hosted site (curl)	Pass	`/health`, `/graph`, `/schedule`, `/classify`, `/route` and `/simulate/advance` all return the right JSON, and wrong input gets a proper 400 error
Browser check on the hosted site	Pass	The map, the truck animation and the database view all load and work

Deliverables Progress

- C++ backend: routing, hazard dispatch, classifier, scheduler, simulation, REST API – **Completed**
- Browser front end with map, route animation, schedule and database views – **Completed**
- Dehradun city data – **Completed**
- Unit tests (4 suites + city data check) – **Completed**
- README and API documentation – **Completed**, including the live link
- Hosted demo (Dockerfile + deployment on Render) – **Completed**
- Phase-II report and presentation – **Completed**
- Phase-III report – **Completed**; Phase-III presentation and final demo – **Pending**